

Republic of the Philippines

Region VI – Western Visayas

CENTRAL PHILIPPINE UNIVERSITY

College Of Computer Studies

Lopez, Jaena St., Jaro, Iloilo



MiniOS Alpha Group 1

In Partial Fulfillment of

IT3131: System Administration and Maintenance

Submitted by:

Pelingon, Bern Ryan M.

Bomban, Thomas Joseph A.

Gatuteo, Ralph Christian C.

Gayorgor, Jamir Adrian J.

Hermonio, Jericko C.

Pineda, Clarisse Marie H.

Submitted to:

A.M.P Latter P. Taasan

Adviser

OCTOBER 2026

Chapter 1

Introduction

MiniOS is a text based-python command-line operating system simulator designed for an Operating Systems course. It provides a command prompt through which a user can practice basic operating-system concepts, including file operations, process tracking, simulated memory allocation, user authentication, and system information.

MiniOS is an educational simulation, not a real operating system. It does not boot a computer, implement a kernel, schedule actual CPU tasks, control physical RAM, or manage Windows processes. Its purpose is to demonstrate the logic and terminology of operating-system services in a small, understandable program.

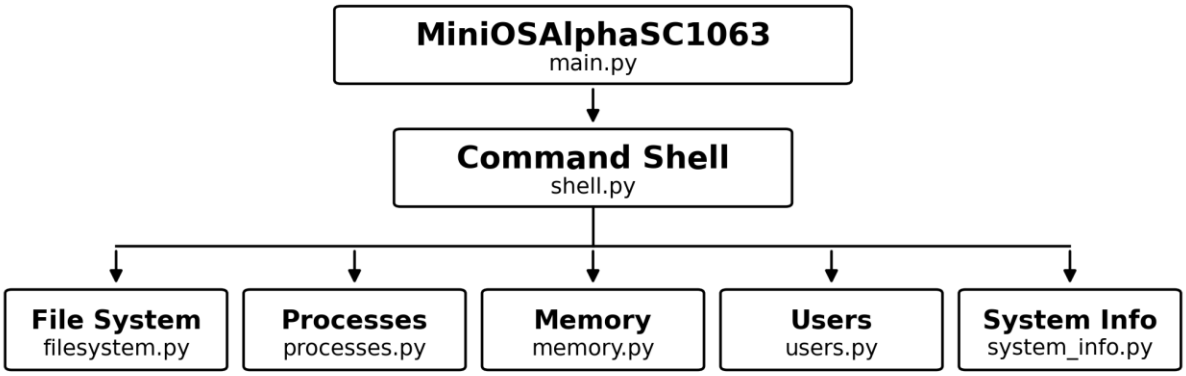
2. Project Objectives

1. Develop a command-line interface that accepts and processes MiniOS commands.
2. Implement basic file and directory operations in a controlled workspace.
3. Represent running processes using simulated process IDs and statuses.
4. Demonstrate memory allocation and release using a fixed virtual memory capacity.
5. Provide a basic login system with at least two demo accounts.
6. Display system and current-user information.
7. Handle invalid commands, missing files, invalid arguments, and resource-limit errors without terminating unexpectedly.
8. Document the system design, operation, testing approach, and known limitations.

3. System Architecture

MiniOS uses a command shell connected to file management, process management, memory management, user management, and system information. The diagram below shows these main parts and the Python files that handle them.

Figure 1 System Architecture.



"main.py" starts the program and checks login through users.py before opening the shell. The shell then calls the module needed for each command. All five modules are separate parts connected to the shell.

The MiniOS architecture relies on six core components to manage its simulated operations. The Command Shell processes user input, checks arguments, and triggers handlers through commands like help, history, and exit. Storage is handled by the File System, which manipulates files and folders inside the MiniOS storage using dir, mkdir, create, read, and delete. For execution and resource management, the Processes component tracks simulated running tasks (accessible via run, process, and kill), while the Memory component monitors total, used, and free simulated RAM using the memory, allocate, and free commands. Access control is driven by the Users component,

which handles accounts and verifies logins through whoami, users, login, logout, and adduser. Finally, System Info fetches the system profile and reads the local computer clock via the systeminfo, date, and time commands.

4. Program Structure

The program is divided into separate Python files. Each file has its own task, which makes the code easier to understand and update. main.py connects the managers, while shell.py handles the commands entered by the user.

1. The 'main.py' is the starting file. show_banner() displays the OS name and version, while main() finds the project folder, loads UserManager, and runs login. After login, it creates the file, process, and memory managers, passes them to MiniOSShell, and starts the shell.
2. The 'shell.py' contains MiniOSShell. start() runs the input loop and records command history. execute() separates the input, finds the matching handler, and checks arguments. Handlers such as _cmd_create() and _cmd_allocate() call the managers. The shell also handles errors and shutdowns.
3. The 'command_catalog.py' contains a Command record for each command, including its category, syntax, purpose, example, and argument limits. BY_NAME allows the shell to find a record by name. help_text() builds the grouped help list, and detail_text() provides help for one command.
4. The config.py keeps OS_NAME, GROUP_NAME, VERSION, and TOTAL_MEMORY_MB in one place. The memory capacity is set to 128 * 1024, or 131,072 MB. format_memory() displays an amount in both MB and GB, using the project's conversion of 1 GB = 1024 MB.

5. The ‘filesystem.py’ contains FileSystemManager. It keeps the storage root and current directory. _safe_path() checks that a path stays inside the storage area. Its methods create, list, read, write, rename, and delete files or folders, while protecting the root and current directory.
6. The ‘processes.py’ contains SimulatedProcess and ProcessManager. Each process record has a PID, name, and status. ProcessManager keeps the records in a dictionary, assigns increasing PIDs, and supports run(), process_table(), and kill(). These methods only change simulated records.
7. The ‘memory.py’ contains MemoryManager. It stores total_memory and used_memory, and calculates free_memory from their difference. allocate() and free() validate amounts before changing used_memory. status_text() prepares the memory display. No actual computer RAM is controlled.
8. The ‘users.py’ contains UserManager. It checks demo and added-account logins and keeps current_user. It also creates accounts, rejects invalid or duplicate names, and saves added accounts in data/users.json. Added passwords are saved as hashes with random salts; demo passwords are defined directly in the file.
9. The ‘system_info.py’ contains SystemInfo. info_text() prepares the OS name, group, version, virtual CPU, architecture, simulated memory capacity, current user, and host Python version. current_date() and current_time() read the computer's local clock. The virtual hardware details describe the simulation.
10. The ‘utils.py’ contains shared helpers. clear_screen() runs cls on Windows to clear the CMD display and is called by the shell's clear command. It also defines helpers for printing titles, errors, and success messages.

The supporting files and folders structure the MiniOS environment and manage its underlying data. The minios_storage/ directory, which generates automatically at startup if missing, holds the text files and folders created by users, while data/users.json securely stores usernames, password hashes, and salts whenever a new account is added. For development and reference, the tests/ folder contain automated scripts including test_minios.py, test_refinements.py, and test_adduser.py, and the docs/ directory houses the user manual, editable guide, and testing notes. Finally, the root directory includes key administrative files: README.md explains the project, demo accounts, and usage instructions; requirements.txt confirms that no third-party Python packages are needed; and. gitignore ensures temporary files and saved account data remain out of normal Git commits.

MiniOS uses Python's standard library. pathlib handles paths, shlex separates commands and arguments, getpass hides passwords, and datetime reads the local date and time. json handles saved account data. hashlib, hmac, and secrets are used for added account password handling. unittest is used for the tests.

5. File Management

If a user wishes to create a folder inside the miniOS, they must use **the mkdir <folder name>** command, which will then create their folder. The user may create files inside the folder by using the **create <file name> [optional text]** command, which creates an empty file. Files can also be created with content immediately existing within it if the user chooses to add text next to the file name before they submit the command, or they may choose the **append <file name> [text]** command after the file has already been made. Write **<file name [text]>** empties the file and replaces the existing text with new text. Both files and folders may also be renamed through the

rename <current file/folder name> <new file/folder name> command. If the user wishes to remove a file, instead, they must use the **delete <file name> for folders.** The **rmdir <directory name> command** is used to remove an empty folder. If a user wants to check the current folder, they're using they must use the **pwd** command, while the **cd** command allows the user to change folders

6. Process Management

Using the **run <command name>** command creates the process with a unique process ID. These processes and their process IDs can be viewed by the user by using the **process** commands. Currently running processes may also be removed or terminated by the **kill <process ID>** command.

7. Memory Management

MiniOSAlphaSC1063 v1.2 simulates memory management using a fixed capacity of 128 GB (131,072 MB). It tracks the total, used, and available memory without using the computer's actual RAM. Users can use the **memory** command to view memory usage, **allocate <MB>** to allocate simulated memory, and **free <MB>** to release it. The system rejects invalid amounts and allocations that exceed available memory. Memory usage resets when the program exits because it is not permanently saved.

8. User Management

Once the miniOS is launched, it will immediately ask the user for a username, which will then be followed by their password. If both are correct, the user will now have access to all the

miniOS features. If the user wants to use another account, they must use the **adduser <chosen username>** command. The miniOS will then ask the user to input a password twice for confirmation. If the user wants to switch to another account, they must first use the **logout** command and repeat the process again. The **users** command displays all existing users in the system.

9. Error Handling

When the user enters a wrong or invalid command, the MiniOS shows an error message that states, "unknown command, type 'help' to see available commands". When you type help the system shows the available commands that can be done by the OS. Also, when you try to create a directory or file that has already been created, the MiniOS returns a reply that says, "A file or directory with that name already exists.". The MiniOS also validates login with a hash password. And lastly, it prevents users from releasing memory that has not been allocated, for example we tried freeing 500mb and the MiniOS returned "Error: Cannot release 500 MB because only 0 MB is allocated".

10. Testing

Our MiniOS was tested by executing the commands available through the CLI/CMD. To start the os we first typed "python main.py", and after that we were directed into the user authentication in which there are 2 users available, the admin and the student. We also tested other commands if it worked like system information display, file management, and memory management. Both valid and invalid inputs were tested to verify whether the system outputs were

valid. After testing it, the results showed that the miniOS could display user information, date and time, system details, command instructions, and can also use calculator. The miniOS showed the displayed error messages for invalid commands, incorrect arguments, invalid login credentials, and unsuccessful release of memory.

11. Problems Encountered

The problems we encountered during the creation and testing of our miniOS is that when you append text, it does not create a space on the previous text. The challenges we have faced includes inconsistent default login credentials, some errors in command syntax, and limitations on file access. The system also struggles with process and memory management since both are simulated. Lastly, certain data, such as process records and command history, is removed when you exit the program. These challenges highlight the need to enhance consistency, validate system functions, and perform extensive testing to ensure the miniOS runs smooth and reliable.

12. Conclusion

Through this project, our group gained a better understanding of how operating concepts work together in a functional system. We learned how different features, such as file management, process management, memory management, and user login, contribute to the system's overall functionality.

We also realized the importance of teamwork, proper testing, and organizing each part of the system. In conclusion, this project helped us strengthen our understanding of operating systems

and improve our problem-solving and collaborative skills. It also gave us a better appreciation of how operating system concepts can be applied in a working program. In addition, it highlighted the importance of designing systems that are organized, reliable, and capable of handling different user inputs and operational scenarios.

13. Screenshots

System Startup and Login

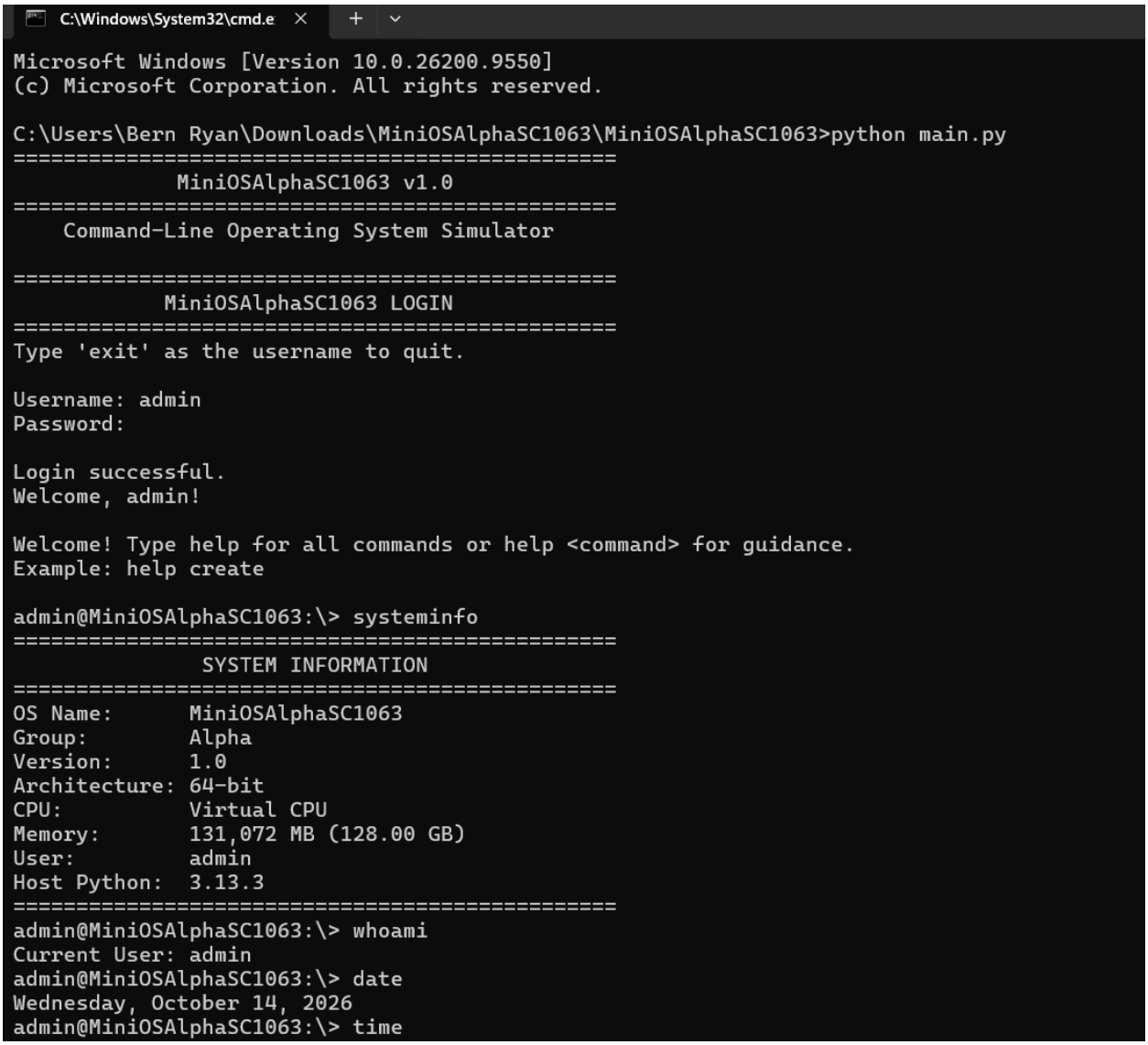


Figure 2.

Help and Command List

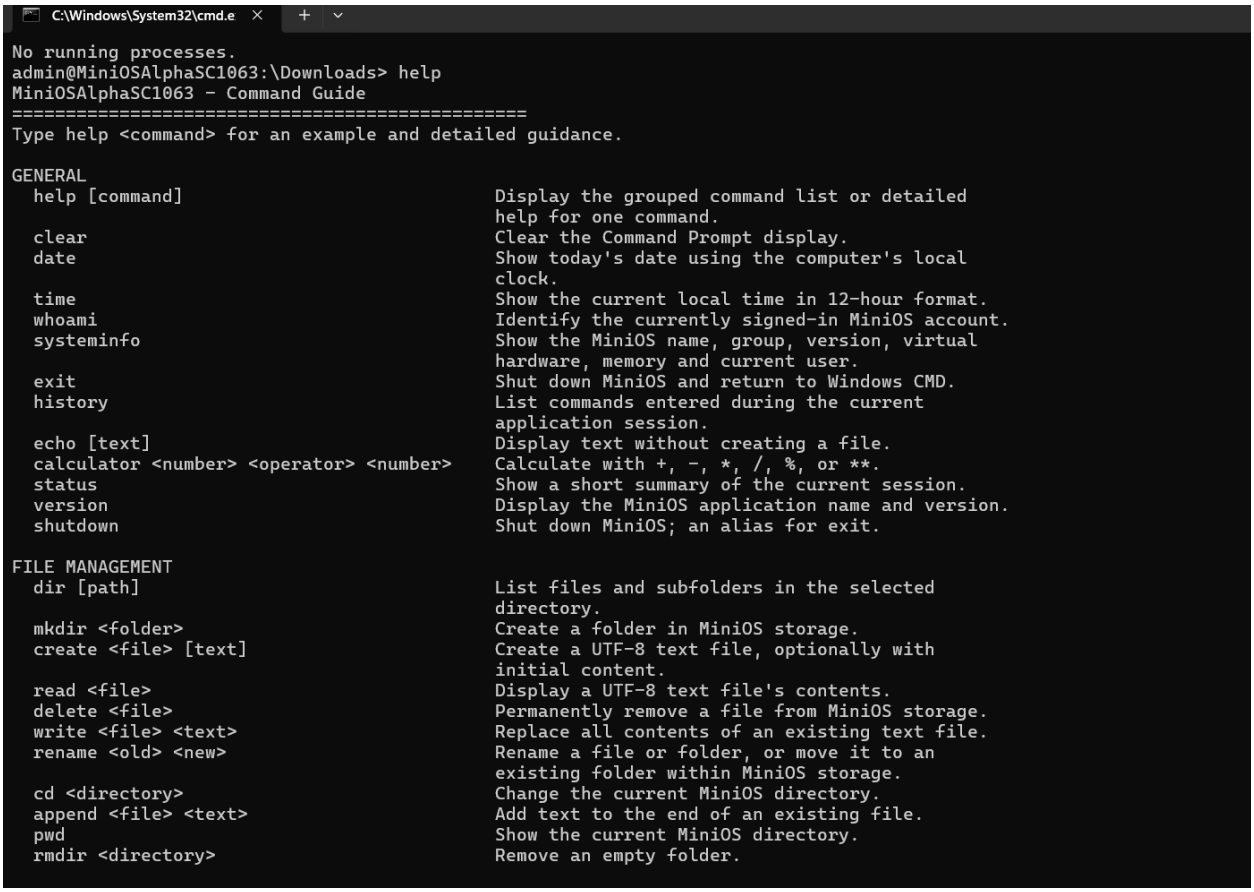


Figure 3.

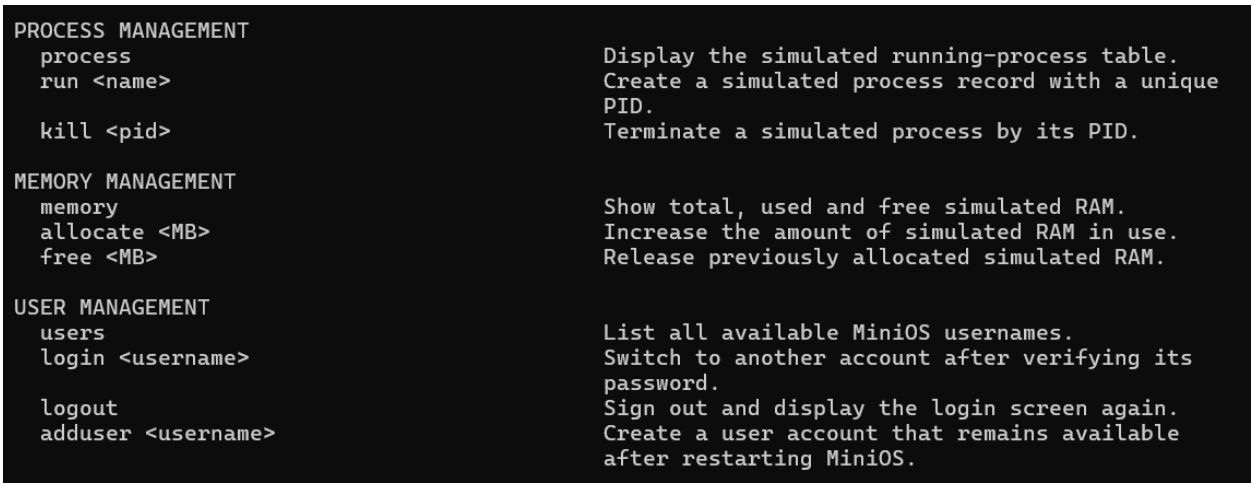


Figure 4.

```
FILE MANAGEMENT
  dir [path]                          List files and subfolders in the selected
                                      directory.
  mkdir <folder>                      Create a folder in MiniOS storage.
  create <file> [text]                Create a UTF-8 text file, optionally with
                                      initial content.
  read <file>                         Display a UTF-8 text file's contents.
  delete <file>                       Permanently remove a file from MiniOS storage.
  write <file> <text>                 Replace all contents of an existing text file.
  rename <old> <new>                  Rename a file or folder, or move it to an
                                      existing folder within MiniOS storage.
  cd <directory>                      Change the current MiniOS directory.
  append <file> <text>                Add text to the end of an existing file.
  pwd                                 Show the current MiniOS directory.
  rmdir <directory>                   Remove an empty folder.
```

Figure 5.

File Management System


```
FILE MANAGEMENT
  dir [path]                          List files and subfolders in the selected
                                      directory.
  mkdir <folder>                      Create a folder in MiniOS storage.
  create <file> [text]                Create a UTF-8 text file, optionally with
                                      initial content.
  read <file>                         Display a UTF-8 text file's contents.
  delete <file>                       Permanently remove a file from MiniOS storage.
  write <file> <text>                 Replace all contents of an existing text file.
  rename <old> <new>                  Rename a file or folder, or move it to an
                                      existing folder within MiniOS storage.
  cd <directory>                      Change the current MiniOS directory.
  append <file> <text>                Add text to the end of an existing file.
  pwd                                 Show the current MiniOS directory.
  rmdir <directory>                   Remove an empty folder.
```


Figure 6.

Memory Management System

```
MEMORY MANAGEMENT
  memory                              Show total, used and free simulated RAM.
  allocate <MB>                       Increase the amount of simulated RAM in use.
  free <MB>                           Release previously allocated simulated RAM.
```

Figure 7.

Process Management System

```
PROCESS MANAGEMENT
  process                             Display the simulated running-process table.
  run <name>                          Create a simulated process record with a unique
                                      PID.
  kill <pid>                          Terminate a simulated process by its PID.
```

Figure 8.

System Information

```
admin@MiniOSAlphaSC1063:\> systeminfo
================================================
                SYSTEM INFORMATION
================================================
OS Name:       MiniOSAlphaSC1063
Group:         Alpha
Version:       1.0
Architecture: 64-bit
CPU:           Virtual CPU
Memory:        131,072 MB (128.00 GB)
User:          admin
Host Python:   3.13.3
================================================
```

Figure 9.

User Management

```
USER MANAGEMENT
  users                                 List all available MiniOS usernames.
  login <username>                      Switch to another account after verifying its
                                        password.
  logout                                Sign out and display the login screen again.
  adduser <username>                    Create a user account that remains available
                                        after restarting MiniOS.
```

Figure 10.

Error Handling

```
admin@MiniOSAlphaSC1063:\Downloads> read more.txt
Error: File does not exist.
```

Figure 12.

```
student@MiniOSAlphaSC1063:\Downloads> hi
Unknown command.
Type 'help' to see the available commands.
student@MiniOSAlphaSC1063:\Downloads> cmd
Unknown command.
Type 'help' to see the available commands.
student@MiniOSAlphaSC1063:\Downloads>
```

Figure 12.